

dev6 全流程分析：基于 tag-test8 收缩产物补充 test0/test6 同类型图表

opencode analy

2026 年 8 月 22 日

摘要

本报告分析一项具体工作(A 代码工作框架):在不重算收缩的前提下,以 $HOME/data/beta6.20_mu-0.2770_ms-0.2400_L24x72$ 下 405 个组态的既有管线产物 (tag-test8 代码生成) 为唯一输入, 调用本库 pyqcd 补充与 examples/test0/v202608150750/plots (3 型) 及 logs/test6/1_result/L24x72/Pz6 (7 型) 相同类型的全部图表。分析范围覆盖 dev6 工作目录全部源文件 (main.py 499 行、verify_dev6.py 110 行、AGENTS.md) 及其调用的 pyqcd 分析模块、两参考目录与运行产物。核心结论: 九类图表全部落地 (test6 型 7 图以 P0/P2 双通道替代 x/y/z/ave 四方向; test0 型 2 图按 docker 风格复现; ratio_3pt 因输入无连通 3pt 数据而诚实跳过并注明根因), 全量运行 CPU 仅 9–12 s、峰值内存 0.44 GB; 双独立方法测得质子能量 $P0 = 1.112-1.147$ GeV (与 stab1 已验证 ≈ 1.12 GeV 结论一致)、 $P2 = 1.491-1.548$ GeV (色散预期 1.510 GeV, 偏差 -1.2% 至 $+2.5\%$), 方法间系统差 35/57 MeV; verify 断言 22 项全部通过。三视角概览: 结构上 dev6 为薄驱动层 (扫描—统计—拟合—成图), 物理计算完全委托 pyqcd.analysis; 关系上它把 tag-test8 的数据层、test0/test6 的图表规范层与 pyqcd 的算法层织成一条只读流水线; 思路坚持”复用已验证实现 + 最小必要修改 + 缺席数据绝不虚构”。

目录

1 仓库概览	2
1.1 规模统计与 git 信息	2
1.2 输入数据布局 (tag-test8 管线产物, docker 布局)	2
2 主体结构	3
3 各部分关系	3
4 项目思路	4
4.1 设计意图	4
4.2 演进脉络	5
4.3 典型工作流	5
5 主题分析	5
5.1 代码—物理对象映射	5

5.2

A1 目的与前期准备

6

5.3

A2 输入是什么

6

5.4

A3 输出应该是什么

6

5.5

A4 整体框架

6

5.6

A5 关键细节

7

5.7

A6 任务如何正确执行

7

5.8

A7 实际执行情况

8

5.9

A8 实际输出情况

8

5.10

A9 结果分析

9

5.11

A10 综合分析

9

5.12

A11 结尾总结

10

5.13

A12 结尾评价

10

5.14

A13 补充说明

10

5.15

A14 参考源

11

5.16

A15 附录与附件

11

6

关键代码/文档片段

11

7

本次大任务图表详览

18

8

本次大任务日志关键内容汇编

18

9

参考源清单

26

10

结论与局限

27

1 仓库概览

本节交代被分析工作所处的仓库背景。任务对象为 PyQCD 仓库（格点 QCD 研究库，核心目标为梯度流重整化方案下的核子胶子 TMD-PDF），本次工作在其示例层新增 dev6 工作目录，输入数据位于仓库外的数据目录（由用户在任务书中显式引入）。

1.1 规模统计与 git 信息

1.2 输入数据布局 (tag-test8 管线产物, docker 布局)

文件（每 conf 一份）	形状	类型	物理含义与本任务消费情况
corr_pp_P0_<cid>.npy	(72,)	float64	静止质子 2pt 关联函数 $C_{P0}(t)$ ，动量 (0,0,0)；消费
corr_pp_P2_<cid>.npy	(72,)	float64	质子 2pt，动量 (0,0,2) ($n_z=2$ ，带相位残留负号)；消费

文件（每 conf 一份）	形状	类型	物理含义与本任务消费情况
VdV_mom_<cid>.npy	(72,2,100,100)	complex64	$V^\dagger V$ 顶点积（双动量槽）；不消费（目标图型不需要）
VVV_mom_<cid>.npy	(72,2,100,100,100)	complex64	VVV 三顶点积；不消费

表 2: 输入数据布局（数据来源: `ls/np.load` 实测；动量定义见 `pyqcd/pipeline/_config.py:51-52` $MOM_SINK_VDV/VVV=[[0, 0, 0], [0, 0, 2]]$ ）

关键取舍：目标图表类型均只依赖 2pt 数据，故驱动层完全不读取单份 1.15 GB 的 VVV 文件——这是低资源约束下最直接的省时省存设计（避免 405×1.15 GB 的无谓 IO）。

2 主体结构

上一节给出了数据底座，本节回答”代码如何分层”。dev6 是一个自包含的薄驱动目录：入口、统计、拟合、成图各司其职，物理算法一律下沉到 pyqcd。

层次	成员	职责
入口层	<code>main.py:432-462</code>	参数解析、组态扫描调度、输出目录编排、汇总 JSON
数据访问层	<code>main.py:84-105</code>	<code>scan_conf_ids</code> 扫描双通道齐全组态； <code>load_raw</code> 堆叠为 (N_{conf}, N_t)
物理统计层	<code>pyqcd.analysis</code> （复用）	<code>resample/sem/cov_mat</code> (delete-one jackknife) 、 <code>run_meff_jackknife</code> (cosh 型有效质量链)
拟合层	<code>main.py:109-161</code>	<code>energy_model</code> 四参数形状模型 + <code>make_p0</code> 数据驱动初值 + <code>do_fit</code> 逐样本 lsqfit
成图层	<code>main.py:228-357, 360-430</code>	<code>test6</code> 型七图 + <code>test0</code> 型两图（docker 风格 2×2 栅格）
守卫层	<code>verify_dev6.py</code>	齐全性/形状/有限性/物理自治 22 项断言
文档层	<code>AGENTS.md</code>	运行方式与关键约定
产物层	<code>v202608221540/</code>	9 图 + <code>corr2.npy</code> + 拟合 <code>npz/report</code> + <code>summary/run.log</code>

表 3: dev6 主体结构分层（数据来源: `grep` 锚点实测）

3 各部分关系

结构与关系互为表里：本节给出依赖主链与三条参照链。

指标	实测值
当前分支 / HEAD	main / b4c75b6 (2026-08-22-14-40-59)
提交数 / 标签数	67 / 36
dev6 源文件行数	main.py 499 + verify_dev6.py 110 + AGENTS.md 39 = 648
本次运行产物数	v202608221540/ 内 15 个文件 (9 PNG + 4 数据 + run.log)
输入组态数	405 (conf4050–conf24300, 步长 50, 四类文件零缺失实测)

表 1: 仓库与工作目录实测统计 (数据来源: wc/ls/git 实测)

要点

运行时依赖主链: *HOME/data/.../conf** / (只读数据层) → main.py (驱动) → pyqcd.analysis (resample/sem/cov_mat/run_meff_jackknife/plot 基元) → gvar+lsqfit (逐样本拟合) → matplotlib (Agg 成图) → v202608221540/ (产物)

三条设计期参照链 (照抄逻辑、不 import 来源, 符合仓库反模式约定):

- **test8 数据规范链:** logs/test8 (makedata/docker 布局约定) → *HOME/data/...* 输入布局 (AGENTS.md 记载 makedata 经 pyqcd 收缩管线自行计算并存 docker 布局);
- **test0 图型 A 链:** examples/test0/v202608150750/plots 三图 ← pyqcd/pipeline/_steps.py: 1030–1115 的 plot_meff_results / plot_correlators / plot_connected_ratio (docker analyze.py 风格);
- **test6 图型 B 链:** logs/test6/1_result/L24x72/Pz6 七图 ← logs/test6/main.py plot_part3 (295–424 行)。

一处必要的库内修改 (最小改动): pyqcd/analysis/_correlators.py:38–40 在通道键缺失时跳过该通道而非抛 KeyError, 使 run_meff_jackknife 可直接服务于仅含质子通道的数据集 (对完整四通道数据的既有行为不变, examples/test0 的 237 项基线断言不受影响)。

4 项目思路

4.1 设计意图

三条意图贯穿始终。其一, **复用而非重算**: 输入已是收缩完成的 2pt, 重跑 vertex/OPE 既无必要也违反分钟量级约束; 其二, **同型适配**: ”同类型图表”指图表语义与版式同型, 数据维度按可得通道映射——B 型的 x/y/z/ave 四方向映射为 P0/P2 双通道 (静止质量点与色散点的物理对照恰好保留), A 型的 2×2 四通道栅格保留版式、空面板如实注明缺数据; 其三, **诚实优先**: 连通 3pt 比值需要 perambulators 数据, 输入集中不存在, 宁可缺席并在 summary/verify/报告三处注明根因, 也不虚构曲线。

4.2 演进脉络

从任务书 (logs/v20260820.txt 的 test9 系列设定) 到 dev6 的演进是”计算链 → 分析链”的自然延伸: test8 解决”算得出”(100 组态正式收缩), test6/test12 风格解决”析得清”(合成数据功能验证), stab1 解决”信得过”(真实数据全链实战), dev6 则在更大统计 (405 组态) 上把分析链图表规范反哺回真实数据集。

4.3 典型 workflow

一次典型运行走遍各层: python main.py → 扫描 405 组态 (双通道齐全性) → load_raw 堆叠 → Part B: resample(jackknife) 截取 $t < 20$ → meff = $\ln |C(t)/C(t+1)|$ → 窗 [6,12] 逐样本 lsqfit → 七图落盘 → Part A: run_meff_jackknife → 两图落盘 → analysis_summary.json → verify_dev6.py 22 断言。

项目思路

一句话: dev6 把”已有数据 + 已验证算法 + 参考图型”三者用最薄的胶水粘合, 让新统计量的产出成本降到分钟级, 同时把不可得之物明确留白。

5 主题分析

本主题为一项具体工作, 按 A 代码工作框架组织 15 步; 物理解读并入相应步骤 (对象映射先行)。上节回答了思路, 本节进入正题: 这项工作如何完整走一遍。

5.1 代码—物理对象映射

代码符号	对象概念	物理/统计含义
corr_pp_P0/P2	质子两点函数 $C_P^{(2)}(t) = \langle \chi_P(t) \bar{\chi}_P(0) \rangle$	零动量/动量 (0,0,2) 投影; 大 t 处 $\sim e^{-Et}$
sgn 约定	相位残留 π	本输入窗口内 $C < 0$; 全局符号不可观测, 拟合 $\text{sgn} \cdot C$
meff = $\ln C/C' $	有效质量 $aE_{\text{eff}}(t)$	单态近似下的逐步能量估计
energy_model	$c_0 e^{-E_0 t} (1 + c_1 e^{-\Delta E t})$	基态 + 单激发两点形状模型
plateau [6,12]	基态平台窗	激发态污染压低区 (对齐 run_meff_jackknife 质子窗)
E0(GeV)	强子能量 (物理单位)	$\text{unit} = 0.197/a = 1.8708 \text{ GeV}$ ($a = 0.1053 \text{ fm}$)
p_phys	格点动量物理值	$2\pi n_z / L \times \text{unit} \approx 0.980 \text{ GeV}$ ($n_z=2$)
R_3pt (缺席)	连通 3pt/2pt 比值	需 perambulators, 输入不含 ⇒ 留白

代码符号	对象概念	物理/统计含义
表 4: 代码符号 ↔ 对象概念映射 (数据来源: main.py/pyqcd 实测对照)		

5.2 A1 目的与前期准备

工作的存在理由：用户要求在既有收缩结果之上补齐两类参考目录的全部同型图表，作为更大统计（405 组态 vs stab1 的 10 组态）下的分析链实战。前置条件有三：输入数据齐全性（实测 405 conf 四类文件零缺失）、Python 环境（numpy/matplotlib/ gvar/lsqfit，均为仓库既有依赖）、pyqcd 可导入（sys.path 注入仓库根，main.py:38-45）。证据：main.py:1-31。

5.3 A2 输入是什么

输入即表 2 所列四类文件；参数块集中于 main.py:55-70：NT=72、NX=24、UNIT=1.8708、CHANNELS=(P0,P2)、DT_MAX=20、拟合窗 [6,12]、svdcut=1e-6、绘图 xlim [-0.5,15.5]。其余未提及设定参照任务书 logs/v20260820.txt（conf 序列、分钟量级子任务约束等，logs/v20260820.txt:1）。

5.4 A3 输出应该是什么

预期产物为十类图表中的九类可落地（映射见 A4 后的类型总表）：B 型七图 + A 型两图，外加 corr2 数组、拟合 npz/report 与汇总 JSON；A3 中 ratio_3pt 应输出但不可算，处理方式为”缺席 + 注明”。

5.5 A4 整体框架

主流程 main() 编排为 load → part_b → part_a → summary 四段（main.py:432-462）；part_b/part_a 各自内聚。目标图表类型映射总表如下——它是”补充所有同型图表”这一验收标准的操作化分解：

参考类型（来源）	dev6 落地	适配说明	状态
eff_mass.png (B)	同名	x/y/z/ave 四系列 → P0/P2 双系列	完成
sem_comparison.png (B)	同名	同上	完成
eff_mass_GeV.png (B)	同名	ave 单系列 + 单色 带 → 双系列 + 各 自 E0 色带	完成
eff_mass_fit_dirs.png(B)	同名	2×2 方向面板 → 1×2 通道面板	完成
corr2_raw.png (B)	同名	log C 四系列 → 双 系列	完成

参考类型（来源）	dev6 落地	适配说明	状态
meff_corr.png (B)	同名	x/y/z 相关系数矩阵 $\rightarrow$ P0/P2 矩阵	完成
meff_hist.png (B)	同名	四方向直方图 $\rightarrow$ 双通道直方图	完成
correlators_all_channels.png (A)	同名	docker 2 $\times$ 2; pion 两面板注明缺失	完成
meff_all_channels.png (A)	同名	同上	完成
ratio_3pt_all_channels.png (A)	—	需连通 3pt/perambulators, 输入不存在	缺席 + 注明

表 5: 目标图表类型映射与可行性判定（数据来源: 参考目录清单 + 输入数据实测）

5.6 A5 关键细节

三个易错点决定了实现的正确性。其一，统计基元直接复用 pyqcd 而非重写：delete-one jackknife 重采样与 sem 的 $\sqrt{N-1}$ 修正、奇异协方差的条件数诊断均来自 `_disconnected.py:23-51`（引文见第 6 节）。其二，逐样本拟合必须共享同一协方差且容许奇异 (svdcut=1e-6) ——这是 test6 在 10 组态协方差奇异实战中沉淀的约定。其三，也是本次唯一的 debug 迭代：首跑全量时 P2 通道拟合出现双峰（中位数 0.830 vs 均值 0.901，约 3.5% 样本漂移到 ~ 1.195 ，误差膨胀到 5.58 GeV、 $\chi^2/\text{dof} = 4.40$ ）。定位过程：检查 E0 样本分布排除离群大值 $\rightarrow$ 检查原始数据发现两通道窗口内 $C(t) < 0$ （相位残留 π ，stab1 已记载“P2 2pt 带 phase 负号”） $\rightarrow$ 正振幅初值落入错误盆地。修复为全局符号约定 $\text{sgn} = \text{sign}\langle C \rangle_{[6,12]}$ 、拟合 $\text{sgn} \cdot C$ (`main.py:127-161`)，修复后 $\chi^2/\text{dof} = 0.61$ 、均值 = 中位数（单峰），P0 亦然 (0.85)。物理依据：全局符号不可观测，不影响 E0；报告头显式声明该约定 (2_fit_report.txt sign convention 行)。

5.7 A6 任务如何正确执行

标准命令序列（低资源要点：纯 CPU、只读小文件、先冒烟后全量）：

```

1 cd examples/pyqcd/dev6
2 python main.py --debug          # 前 5 组态冒烟 -> 0_debug/
3 python main.py                  # 全量 405 组态 -> v<ts>/
4 python verify_dev6.py <run_dir> # 22 项断言门

```

实测耗时与资源：冒烟（5 组态）8.8 s；全量 11.5 s；复跑落盘 run.log 时 9.1 s（确定性管线，数值逐位一致）；全程峰值内存 0.439 GB、纯 CPU。

5.8 A7 实际执行情况

实际执行完全按 A6 序列推进，期间发生一次 debug 迭代（P2 拟合双盆地形，见 A5，修复后重跑）。运行实录关键行（完整见 L2 日志汇编）：

```

1 P0: E0=1.1119(8) GeV chi2/dof=0.85 cond=1.64e+05 (2.6s)
2 P2: E0=1.4911(52) GeV chi2/dof=0.61 cond=2.94e+03 (2.7s)
3 proton P[0, 0, 0]: E0 = 1.1474 +- 0.0015 GeV (expected 1.000, plateau t in [6,12])
4 proton P[0, 0, 2]: E0 = 1.5477 +- 0.0027 GeV (expected 1.510, plateau t in [6,12])
5 pion P0: 输入缺少 corr_pion_P0 -- 跳过该通道
6 Peak Memory: 0.439 GB / total time: 9.1s / job finish

```

证据：[v202608221540/run.log](#) (全文直引见第 8 节)；pion 跳过消息即 `pyqcd/analysis/_correlators.py:38-40` 新增守卫的实际触发记录。

5.9 A8 实际输出情况

产物清单 (15 文件)：九图 (F1–F9)、`corr2_P0/P2.npy` 各 (405, 20) float64、`1_fit_data.npz` (每通道 `c0/c1/E0/dE/chi2` 五数组 $\times$ 405 样本，405/405 全部收敛、零 NaN)、`2_fit_report.txt` (汇总表：P0 $E_0 = 1.112(8)$ GeV、P2 $E_0 = 1.491(52)$ GeV)、`analysis_summary.json` (A/B 双方法数值 + 缺席图表根因字段)、`run.log`。拟合报告原文直引：

```

1 =====
2   Fit Report   : dev6 L24x72 (proton P0/P2)
3   =====
4   dt range      : [6, 12]
5   unit          : 1.8708 GeV (a=0.1053 fm)
6   svdcut        : 1e-06
7   jackknife     : True
8   sign convention: fit sgn*C (窗口内 C<0, 全局符号不可观测)
9   =====
10
11  -----
12  channel = P0, condition number = 1.64e+05, Nfit = 405/405, sgn = -1
13
14   P0  c0=0.202(8)  E0=0.594(4)  chi2/dof=0.85
15  -----
16  channel = P2, condition number = 2.94e+03, Nfit = 405/405, sgn = -1
17
18   P2  c0=0.0664(23)  E0=0.797(28)  chi2/dof=0.61
19  =====
20   Summary Table (E0 in lattice & GeV)
21  =====
22  +-----+-----+-----+-----+-----+
23  | channel | E0(a^-1) | E0(GeV) | c0 | chi2/dof |
24  +-----+-----+-----+-----+-----+
25  |   P0   | 0.594(4) | 1.112(8) | 0.202 | 0.85 |
26  |   P2   | 0.797(28) | 1.491(52) | 0.0664 | 0.61 |
27  +-----+-----+-----+-----+-----+

```


5.10 A9 结果分析

以下逐图解读全部九图（图件本体集中于第 7 节，此处给物理判读）。

F1 eff_mass.png (aE 对比)：双通道有效质量在 $t \geq 5$ 后均进入清晰平台，P0 平台值约 0.59 (a^{-1} 单位)，P2 约 0.80，末点因周期性卷绕上翘属正常伪影（参考 test6 同款图的末点行为）。平台分离直观体现动量注入。

F2 sem_comparison.png (SEM 散点)：两通道 SEM 随 t 增大而增大（信号衰减所致）， $t < 12$ 内 P2 的 SEM 略大于 P0，量级 10^{-3} – 10^{-2} ，与 jackknife 统计自洽；该图用于拟合窗选择的误差预算依据。

F3 eff_mass_GeV.png (GeV + 双色带)：换算物理单位后，灰色带给出 B 型拟合 $E_0^{P0} = 1.112$ GeV 与 $E_0^{P2} = 1.491$ GeV 及其误差带宽；曲线在窗内与色带相切，表明形状模型对平台段描述良好 ($\chi^2/\text{dof} = 0.85/0.61$ 均接近 1)。

F4 eff_mass_fit_dirs.png (逐通道面板)： 1×2 面板各自独立拟合，P2 面板早期 t 点略高于色带——单激发残留的典型形态，正是拟合模型引入 $c_1 e^{-\Delta E t}$ 项要吸收的部分。

F5 corr2_raw.png (log|C|)：log 半负定直线在 $t \in [2, 12]$ 近似线性，斜率即 $-E$ ；P0/P2 斜率差可见。 $t > 40$ 后进入噪声底（图中 xlim 截至十五，长尾在 corr2.npy 中保留供检查）。

F6 meff_corr.png (相关系数矩阵)： $t = 9$ 处 P0–P2 meff 相关系数显著为正（同系综组态涨落共享），条件数标注提示协方差良构；此矩阵是未来多动量联合拟合（如色散关系约束拟合）的输入级信息。

F7 meff_hist.png (组态直方图)：双通道在 $t = 9$ 的逐组态 meff 分布近似高斯、无多峰与长尾——这是 405 组态统计下“单盆地收敛”的最直接证据，也反证首跑双峰确系拟合盆地问题而非数据本身多模态。

F8 correlators_all_channels.png (A 型关联函数栅格)：左列/上行两个质子面板 log 纵轴直线下降、 $C(0)$ 数量级正确；右下两面板如实注明“no data (channel absent in input)”——版式保持 docker 四面板风格，信息诚实。

F9 meff_all_channels.png (A 型有效质量栅格)：质子两面板含平台阴影区 $[6, 12]$ 、 E_0 虚线与 E_{exp} 点线；标题 dev 值偏大的原因见 A10 第三条（基线粗期望约定），不影响测量值本身。

拟合报告（A8 引文）补充两点：cond 分别为 $1.6 \times 10^5 / 2.9 \times 10^3$ （svdcut 截断下稳定）；Nfit = 405/405 表明符号约定修复后无坏样本。

5.11 A10 综合分析

三条综合判断。第一，**双方法互证**：B 型（相关协方差四参数形状拟合）与 A 型（cosh 有效质量非相关平台加权）是两条几乎独立的估计路径，结果为

通道	B 型 E_0 [GeV]	A 型 E_0 [GeV]	差值与方法解读
P0	1.1119(8), $\chi^2/\text{dof} = 0.85$	1.1474(15), plateau[6,12]	35 MeV: 窗内形状弯曲的系统差
P2	1.4910(28), $\chi^2/\text{dof} = 0.61$	1.5477(27)	57 MeV: 同上 + P2 激发态残留

通道	B 型 E_0 [GeV]	A 型 E_0 [GeV]	差值与方法解读
----	-----------------	-----------------	---------

表 6: 双独立方法能量对照 (数据来源: analysis_summary.json 实测)

第二, **色散关系校验**: $p = 0.980$ GeV, 以 A 型 $m_0 = 1.147$ GeV 代入 $E = \sqrt{m_0^2 + p^2} = 1.510$ GeV, A 型 P2 偏 +2.5%、B 型偏 -1.2%, 等效 $k_2 \simeq O(1)$, 与该系综族既有结论一致——P2 数据确实位于色散曲线上, 而非离群。第三, **dev 标志的正确读法**: A 型日志中 P0 "dev=101 σ " 并非异常: 基线约定 $E_{\text{exp}}(P0) = 1.0$ 是 docker analyze.py 的粗期望常数而非物理预言; 405 组态使统计误差压到 1.5 MeV, 任何 ~MeV 级系统差都会放大 σ 计数。真正的物理校验是与 stab1 已验证结论对比: 本输入 P0 测值 1.112–1.147 GeV 与 docker 基线质子质量 ≈ 1.12 GeV 同量级相符 (不同组态子集, 推断级)。第四, **对象映射回填**: sgn 约定的引入把"数据相位惯例"显式化为可配置项, 若未来输入改为相位校正后的数据 (如 test6 的 eginphase2 命名系列), sgn 自动为 +, 代码无需改动——映射表第 2 行因此具有前向兼容性。

5.12 A11 结尾总结

结论

dev6 以 648 行驱动代码 + 一次库内三行守卫修改, 在只读 405 组态既有收缩产物的前提下, 分钟级 (9–12 s CPU、0.44 GB 内存) 补齐两类参考目录的全部可落地同型图表 (9 类), 缺席的连通 3pt 比值以三处联署方式留白; 双方法能量互差 35/57 MeV, P0 与 stab1 已验证 ≈ 1.12 GeV 结论一致, P2 与色散预期偏差 $\leq 2.5\%$; 22 项 verify 断言全部通过。

5.13 A12 结尾评价

优点: 复用率高 (统计/拟合/A 型链全部复用 pyqcd, 新增代码集中在 I/O 与成图胶水); 可复现性强 (run.log + summary.json + report 三重记录, 确定性管线逐位复现); 低资源 (不触碰 VVV 大文件)。可改进处: B 型与 A 型的平台窗目前同为 [6,12] 但方法论上允许各自优化; meff_corr 仅双通道, 未来多动量数据到位后矩阵才有完整意义; ratio_3pt 待 perambulators 数据就绪后可在同一框架下补算。

5.14 A13 补充说明

假设与局限: 假设输入 corr_pp 已含正确的源汇投影与相位约定 (仅残留全局符号); 假设 405 组态为同系综同参数独立样本; 未验证项包括 pion 通道、连通/不连通比值、以及 $t > 20$ 区域的系统性质 (DT_MAX 截断仅为成图窗口)。

局限与风险

风险点：(1) ratio_3pt_all_channels.png 缺席——根因是输入数据集不含 perambulators/3pt 产物，待数据补齐后重跑；(2) A 型 E_{exp} 为基线硬编码粗期望，dev 值不具假设检验含义（已按约定继承并注明）；(3) P2 窗口内激发态残留在 F4 可见，若需更紧的能量误差应做窗变系统性分析（未验证，列为后续项）。

5.15 A14 参考源

本步全部证据已在正文各处以 evidence 绿标注，集中清单见第 9 节（共 22 条）。

5.16 A15 附录与附件

原始附件随产物保存：run.log（运行实录全文，第 8 节直引）、2_fit_report.txt（A8 直引）、analysis_summary.json（第 8 节直引关键段）、会话日志.all.2026-08-22-15-01-24.log（第 8 节直引）。本报告 docs/ 目录即 analy 技能唯一写操作。

6 关键代码/文档片段

以下片段一律以 listings 的 lstinputlisting 命令直引仓库原文件，保证与仓库逐字节一致。
参数块（单位约定与窗口常数的单一事实来源）：

```

1
2 # ===== 参数块（对齐 logs/v20260820.txt 与 pyqcd/pipeline/_config.py 约定） =====
3 DATA_ROOT = os.path.join(
4     os.environ.get("HOME", "/public/home/zhangxin"), "data",
5     "beta6.20_mu-0.2770_ms-0.2400_L24x72")
6 NT = 72
7 NX = 24
8 ALTTc = 0.1053                # 格距 fm (_config.py)
9 UNIT = 0.197 / ALTTc          # 格点能 → GeV (test6 约定, 约 1.8708)
10 CHANNELS = ("P0", "P2")      # 动量标签: (0,0,0) 与 (0,0,2)
11 DT_MAX = 20                  # 分析窗长 (test6 约定)
12 FIT_START, FIT_END = 6, 12    # 平台拟合窗 (对齐 run_meff_jackknife 质子窗)
13 DT_REF = (FIT_START + FIT_END) // 2
14 SVD_CUT = 1e-6
15 XLIM = [-0.5, 15.5]
16 YLIM_AE = [0.4, 1.05]        # aE (P0 约 0.60 / P2 约 0.83)

```

组态扫描与加载（双通道齐全性守卫）：

```

1 def scan_conf_ids(data_root):
2     """扫描同时具备 corr_pp_P0/P2 的组态（排序去重）。"""
3     ids = set()
4     for f in glob.glob(os.path.join(data_root, "conf*", "corr_pp_P0*.npz")):
5         cid = int(os.path.basename(f).split("_")[-1].split(".")[0])
6         if os.path.exists(os.path.join(
7             data_root, f"conf{cid}", f"corr_pp_P2_{cid}.npz")):
8             ids.add(cid)
9     return sorted(ids)

```

```

10
11
12 def load_raw(data_root, conf_ids):
13     """读两通道 2pt → {ch: (Nconf, Nt) 实数组}。"""
14     raw = {}
15     for ch in CHANNELS:
16         arr = np.zeros((len(conf_ids), NT))
17         for i, cid in enumerate(conf_ids):
18             arr[i] = np.load(os.path.join(
19                 data_root, f"conf{cid}", f"corr_pp_{ch}_{cid}.npy"))
20         raw[ch] = arr
21     return raw

```

拟合模型与数据驱动初值：

```

1 def energy_model(x, p):
2     """ $C(t) = c_0 e^{(-E_0 t)} (1 + c_1 e^{(-dE t)})$ 。"""
3     dt = np.asarray(x, dtype=np.float64)
4     return (p["c0"] * np.exp(-p["E0"] * dt)
5             * (1 + p["c1"] * np.exp(-p["dE"] * dt)))
6
7
8 def make_p0(corr_mean):
9     """由平台数据构造拟合初值（避免硬编码先验）。"""
10    tw = slice(FIT_START, FIT_END + 1)
11    e_est = np.log(np.abs(corr_mean[FIT_START])
12                  / np.abs(corr_mean[FIT_END - 1])) / max(FIT_END - 1 - FIT_START, 1)
13    c_est = np.mean(np.abs(corr_mean[tw])
14                  / np.exp(-e_est * np.arange(FIT_START, FIT_END + 1)))
15    return {"c0": float(max(c_est, 1e-8)), "c1": 0.1, "E0": float(e_est),
16            "dE": 0.3}

```

do_fit：逐样本 lsqfit + 全局符号约定（debug 迭代的核心修复）：

```

1 def do_fit(corr2, sgn=1.0, verbose=False):
2     """逐样本 lsqfit 平台拟合 → ({name: 样本数组}, cond)。坏样本 NaN 不中断。
3
4     sgn: 全局符号约定（本输入两通道窗口内  $C < 0$ ，相位残留  $\pi$ ；拟合  $\text{sgn} * C$  使
5      $c_0 > 0$  落入物理盆地——全局符号不可观测量，不影响  $E_0$ ）。
6     """
7     import gvar as gv
8     import lsqfit
9
10    n_sample = corr2.shape[0]
11    x_coor = list(range(FIT_START, FIT_END + 1))
12    names = ["c0", "c1", "E0", "dE"]
13    sub_sample = corr2[:, FIT_START:FIT_END + 1] * sgn
14    cov, cond = cov_mat(sub_sample, True)
15    p0 = make_p0(np.abs(corr2.mean(0)))
16
17    result = {name: np.full(n_sample, np.nan) for name in names}
18    result["chi2"] = np.full(n_sample, np.nan)
19    n_bad = 0
20    for i in range(n_sample):
21        try:
22            y_coor = gv.gvar(sub_sample[i], cov)

```

```

23         fit = lsqfit.nonlinear_fit(data=(x_coor, y_coor), p0=p0,
24                                   fcn=energy_model, svdcut=SVD_CUT)
25         for name in names:
26             result[name][i] = fit.pmean[name]
27             result["chi2"][i] = fit.chi2 / fit.dof
28         except Exception:
29             n_bad += 1
30         if verbose and (i + 1) % 100 == 0:
31             print(f"_{i+1}/{n_sample}")
32     if n_bad:
33         print(f"[warn]拟合失败_{n_bad}/{n_sample}个样本 (NaN填充)")
34     return result, cond

```

Part B 编排 (resample → meff → 拟合 → 报告):

```

1 def part_b(raw, out_dir):
2     """B 型: resample → meff → 逐样本拟合 → 7 图 + npz/report."""
3     print("=====Part_B (test6_型能量链) =====")
4     corr2 = {ch: resample(raw[ch][:, :DT_MAX], jackknife=True).real
5               for ch in CHANNELS}
6     mass = {ch: np.log(np.abs(corr2[ch]) / np.abs(np.roll(corr2[ch], -1, axis=1)))
7             for ch in CHANNELS}
8     fits, conds, signs = {}, {}, {}
9     for ch in CHANNELS:
10         t0 = time.perf_counter()
11         sgn = float(np.sign(corr2[ch][:, FIT_START:FIT_END + 1].mean())) or 1.0
12         signs[ch] = sgn
13         fits[ch], conds[ch] = do_fit(corr2[ch], sgn=sgn)
14         r = fits[ch]
15         m = np.nanmean(r["E0"]) * UNIT
16         e = sem(r["E0"][np.isfinite(r["E0"])], True) * UNIT
17         print(f"{ch}:_E0={m:.4f}({e*_1e3:.0f})_GeV_{sgn}"
18               f"chi2/dof={np.nanmean(r['chi2']):.2f}_{sgn}"
19               f"cond={conds[ch]:.3g}_{sgn}({time.perf_counter()-t0:.1f}s)")
20         np.save(os.path.join(out_dir, f"corr2_{ch}.npz"), corr2[ch])
21
22     np.savez(os.path.join(out_dir, "1_fit_data.npz"),
23              **{f"{k}_{ch}": v for ch in CHANNELS for k, v in fits[ch].items()})
24     write_report(fits, conds, signs, out_dir)
25     plot_part_b(corr2, mass, fits, out_dir)
26     return corr2, mass, fits

```

eff_mass_GeV 双色带实现 (B 型图 3 的适配版式):

```

1 # 图 B4: eff_mass_GeV.png (GeV + 双通道各自 E0 色带)
2 fig, ax = plt.subplots(figsize=(10, 6), dpi=150)
3 band_x = np.array([FIT_START, FIT_END])
4 for k, ch in enumerate(CHANNELS):
5     r = fits[ch]
6     ok = np.isfinite(r["E0"])
7     e0 = np.nanmean(r["E0"]) * UNIT
8     e0e = sem(r["E0"][ok], True) * UNIT
9     c = DEFAULT_PLOT_COLORS[k]
10    ax.errorbar(x_vals + k * X_OFFSET, mass[ch].mean(0) * UNIT,
11                yerr=sem(mass[ch], True) * UNIT, fmt="x", color=c,
12                ecolor=c, capsize=0, label=f"{ch}")

```

```

13     ax.fill_between(band_x, [e0 - e0e] * 2, [e0 + e0e] * 2,
14                     color="gray", alpha=0.35, linewidth=0,
15                     label=f"E0[{ch}]=\{e0:.3f}\}")
16     ax.axhline(e0, color=c, ls="--", lw=0.8)
17 chi2s = ",\ ".join(
18     f"\{ch}=\{np.nanmean(fits[ch]['chi2']):.2f}\}" for ch in CHANNELS)
19 ax.set_xlim(XLIM)
20 ax.set_ylim(YLIM_GEV)
21 ax.set_xlabel("t/a")
22 ax.set_ylabel("eff_\mass_\(GeV)")
23 ax.set_title(f"\{title}\nchi2/dof:\{chi2s}\}")
24 ax.legend(loc="upper\_right", fontsize=9)
25 fig.tight_layout()
26 fig.savefig(os.path.join(out_dir, "eff\_mass\_GeV.png"), bbox_inches="tight")
27 plt.close(fig)
28 print("eff\_mass\_GeV.png\_done")

```

Part A: 调用 pyqcd 有效质量链 + docker 风格栅格:

```

1 def part_a(raw, conf_ids, out_dir):
2     """run_meff_jackknife → correlators/meff all_channels (docker 风格) . """
3     print("=====\_Part\_A (test0\_型\_all\_channels) =====")
4     corr_2pt_all = {cid: {"corr_pp_P0": raw["P0"][i],
5                           "corr_pp_P2": raw["P2"][i]}
6                     for i, cid in enumerate(conf_ids)}
7     t0 = time.perf_counter()
8     meff_res = run_meff_jackknife(corr_2pt_all, conf_ids, NT=NT, ALttc=ALTTc)
9     print(f"run_meff_jackknife\_done_\(\{time.perf_counter()\}_\t0:.1f\} s)")
10
11     import matplotlib
12     matplotlib.use("Agg")
13     import matplotlib.pyplot as plt
14
15     grid = [("proton", "P0"), ("proton", "P2"),
16            ("pion", "P0"), ("pion", "P2")]
17     have = {f"\{particle\}_\{mom\}" for particle, mom in grid} & set(meff_res)
18
19     # A1: correlators_all_channels.png (|C| log-y, docker 风格 2x2)
20     fig, axes = plt.subplots(2, 2, figsize=(12, 9))
21     for ax, (particle, mom) in zip(axes.ravel(), grid):
22         res = meff_res.get(f"\{particle\}_\{mom\}")
23         if res is None:
24             ax.text(0.5, 0.5, f"\{particle\}_\{mom\}\nno\_data\n(channel\_absent\_in\_input)",
25                    ha="center", va="center", transform=ax.transAxes,
26                    fontsize=11, color="gray")
27             continue
28         c, ce = res["corr_mean"], res["corr_err"]
29         t = np.arange(len(c))
30         ax.errorbar(t, np.abs(c), yerr=ce, fmt=".", ms=4, capsize=0)
31         ax.set_yscale("log")
32         ax.set_title(f"\{particle\}_P=\{mom\}_C(0)=\{c[0]:.4e\}")
33         ax.set_xlabel("t")
34         ax.set_ylabel("|C(t)|")
35         ax.grid(alpha=0.3, which="both")
36     fig.suptitle(f"2pt\_correlators_\(Jackknife\_mean,\_len(conf\_ids)\_configs)")
37     fig.tight_layout()

```

```
38 fig.savefig(os.path.join(out_dir, "correlators_all_channels.png"), dpi=150)
```

pyqcd 库内最小修改 (缺失通道跳过守卫):

```
1 ]
2 results = {}
3 for particle, mom, key in channels:
4     if key not in corr_2pt_all[conf_ids[0]]:
5         logger(f"{particle}_{mom}: 输入缺少_{key}——跳过该通道")
6         continue
7     ml = f"P[{list(ANALYSIS_MOMENTA[particle].values())[0 if mom=='P0' else 1}]"
8     stack = np.stack([np.real(corr_2pt_all[cid][key]) for cid in conf_ids])
9     jk = Jackknife(stack, Nconf_axes=0)
10    mf = meff(jk['data_sample'], ALttc, Nconf_axes=0, Nt_axes=1,
11              meff_type=meff_types[particle])
```

复用的统计基元 (sem/resample/cov_mat):

```
1 def sem(data, jackknife=True):
2     """样本轴 (axis 0) 均值的标准误。"""
3     error = data.std(0)
4     if jackknife:
5         error = error * np.sqrt(data.shape[0] - 1)
6     return error
7
8
9 def resample(corr, jackknife=True, Nsample=None, seed=0):
10    """Delete-one jackknife (或 bootstrap) 重采样: 样本轴移到 axis 0。"""
11    n_conf = corr.shape[0]
12    if jackknife:
13        return (n_conf * corr.mean(0) - corr) / (n_conf - 1)
14    rng = np.random.default_rng(seed=seed)
15    idx = rng.integers(0, n_conf, size=(Nsample, n_conf))
16    return corr[idx].mean(1)
17
18
19 def cov_mat(arr, jackknife=True):
20    """Jackknife 协方差 (均值) 与特征值条件数。"""
21    diff = arr - arr.mean(0)
22    n = arr.shape[0]
23    if jackknife:
24        cov = np.matmul(diff.T, diff) / n * (n - 1)
25    else:
26        cov = np.matmul(diff.T, diff) / n
27    eig = np.linalg.eigvalsh(cov)
28    cond = eig[-1] / eig[0] if eig[0] > 0 else np.inf
29    return cov, cond
```

B 型七图的参照原型 (logs/test6/main.py plot_part3 开头):

```
1 def plot_part3(corr2: dict, mass: dict, fits: dict, fitpa: FitParams,
2               sampa: SampleParams, plotpa: PlotParams, out_dir: str,
3               jack: bool, verbose=True):
4     if verbose:
5         print("=====plot_start=====")
6     time0 = time.perf_counter()
```

```

7   t0 = time.perf_counter()
8
9   x_vals = np.arange(sampa.dt_max)
10  eff_data = {_dir: (mass[_dir].mean(0), sem(mass[_dir], jack))
11              for _dir in ("x", "y", "z", "ave")}
12
13  title = (f"{sampa.conf_short}, P=({sampa.Px},{sampa.Py},{sampa.Pz}), "
14          f"Nconf={sampa.Nconf}, Nsample={sampa.Nsample}")
15
16  # 图1: eff mass 对比 (原版, aE 单位)
17  plot_errbar(x_vals, eff_data,
18              save_path=os.path.join(out_dir, "eff_mass.png"),
19              xlabel="t/a", ylabel="aE",
20              xlim=plotpa.xlim, ylim=plotpa.ylim,
21              x_offset=plotpa.x_offset, title=title)
22  print(f"eff_mass.png done ({time.perf_counter()-t0:.1f}s)")
23
24  # 图2: SEM 对比散点图 t=0-14 (原版)
25  t_max_sem = 15
26  sem_scatter = {_dir: sem(mass[_dir], jack)[:t_max_sem]
27                for _dir in ("x", "y", "z", "ave")}
28  plot_scatter(np.arange(t_max_sem), sem_scatter,
29               save_path=os.path.join(out_dir, "sem_comparison.png"),
30               xlabel="t/a", ylabel="SEM(aE)",
31               xlim=plotpa.xlim, ylim=plotpa.sem_ylim,
32               x_offset=plotpa.x_offset, title=title)
33  print(f"sem_comparison.png done ({time.perf_counter()-t0:.1f}s)")

```

A 型三图的参照原型 (pyqcd 管线 plot_meff_results 开头): (第 1030–1049 与 1051–1059 行; 第 1050 行标题串含 $\pm$ 字形, 为字体一致性略去)

```

1  def plot_meff_results(meff_results, run_dir, logger):
2      import matplotlib
3      matplotlib.use('Agg')
4      import matplotlib.pyplot as plt
5      pdir = os.path.join(run_dir, 'plots')
6      os.makedirs(pdir, exist_ok=True)
7      channels = [('proton', 'P0'), ('proton', 'P2'),
8                  ('pion', 'P0'), ('pion', 'P2')]
9      fig, axes = plt.subplots(2, 2, figsize=(12, 9))
10     for ax, (particle, mom) in zip(axes.ravel(), channels):
11         res = meff_results.get(f'{particle}_{mom}')
12         if res is None:
13             continue
14         m, e = res['meff_mean'], res['meff_err']
15         t = np.arange(len(m))
16         ps, pe = res['plateau']
17         ax.errorbar(t, m, yerr=e, fmt='o', ms=4, capsize=2)
18         ax.axvspan(ps, pe - 1, alpha=0.15, color='C1')
19         ax.axhline(res['E0'], color='C3', ls='--', lw=1)
20         ax.axhline(res.get('E_exp', 0), color='C4', ls=':', lw=1)

```

```

1         f'(exp_{res.get("E_exp", 0):.2f})')
2         ax.set_xlabel('t'); ax.set_ylabel(r'$m_{\rm eff}$ [GeV]')
3         ax.grid(alpha=0.3)

```



```

4     fig.suptitle('Effective_masses_(Jackknife,10_configs)')
5     fig.tight_layout()
6     out = os.path.join(pdir, 'meff_all_channels.png')
7     fig.savefig(out, dpi=150)
8     plt.close(fig)
9     _info(logger, f"Saved_{out}")

```

verify 断言门核心 (齐全性 + 物理自治):

```

1  B_PLOTS = ["eff_mass.png", "sem_comparison.png", "eff_mass_GeV.png",
2            "eff_mass_fit_dirs.png", "corr2_raw.png", "meff_corr.png",
3            "meff_hist.png"]
4  A_PLOTS = ["correlators_all_channels.png", "meff_all_channels.png"]
5  SKIP_PLOTS = {"ratio_3pt_all_channels.png":
6               "需连通3pt/perambulators数据, 输入数据集中不存在"}
7
8  FAILS = []
9
10
11 def check(name, ok, detail=""):
12     tag = "PASS" if ok else "FAIL"
13     print(f"[{tag}]_{name}" + (f"_{detail}" if detail else ""))
14     if not ok:
15         FAILS.append(name)
16
17
18 def main(run_dir):
19     summary_path = os.path.join(run_dir, "analysis_summary.json")
20     with open(summary_path) as f:
21         s = json.load(f)
22         nconf = s["Nconf"]
23
24     # A. 图表齐全性
25     for p in B_PLOTS + A_PLOTS:
26         fp = os.path.join(run_dir, p)
27         check(f"plot:{p}", os.path.isfile(fp) and os.path.getsize(fp) > 1000,
28             f"{os.path.getsize(fp)}_bytes" if os.path.isfile(fp) else "missing")
29     for p, why in SKIP_PLOTS.items():
30         notes = "_".join(s.get("plots_expected_missing", []))
31         check(f"skip-justified:{p}",
32             (not os.path.exists(os.path.join(run_dir, p))) and p in notes,
33             f"缺席且已注明根因: {why}")
34
35     # B. 数据形状与有限性
36     for ch in CHANNELS:
37         fp = os.path.join(run_dir, f"corr2_{ch}.npz")
38         a = np.load(fp)
39         check(f"shape:corr2_{ch}", a.shape == (nconf, DT_MAX), str(a.shape))
40     z = np.load(os.path.join(run_dir, "1_fit_data.npz"))

```

7 本次大任务图表详览

本次工作产生的全部九张图在此集中收录 (清单闭合, 无遗漏; 生成方式均为 main.py 经 matplotlib Agg 后端, dpi 150, 数据来源 v202608221540/). 各图的物理解读已在 A9 给出, 此处保留原件备查。

编号	图表文件	详尽介绍 (生成方式/含义/解读/关联)
F1	v202608221540/eff_mass.png	plot_errbar 双系列; aE(t) 平台图; P0≈0.59、P2≈0.80; 解读见 A9-F1; 支撑 E0 主结论
F2	.../sem_comparison.png	plot_scatter; SEM(aE) 误差预算; $t < 12$ 内 10^{-3} – 10^{-2} ; 关联拟合窗选择
F3	.../eff_mass_GeV.png	自定义单轴双色带; GeV 化主结果图; B 型 E0 及其误差带; 关联 T4 对照表
F4	.../eff_mass_fit_dirs.png	1×2 通道面板; 各自独立拟合带; P2 早 t 残留可见; 关联 warnbox 第 3 条
F5	.../corr2_raw.png	log C(t) 原始对照; 斜率即 $-E$; 关联输入质量评估
F6	.../meff_corr.png	imshow coolwarm; t=9 通道间相关系数 + 条件数; 关联未来联合拟合
F7	.../meff_hist.png	双通道直方图 @ t=9; 单峰近高斯; 反证首跑双峰为盆地问题
F8	.../correlators_all_channels.png	docker 风格 2×2 log 图; pion 面板如实留白; 关联 A3 类型映射
F9	.../meff_all_channels.png	docker 风格 2×2 平台/E0/E_exp; dev 读法见 A10 第三条

表 7: 本次大任务图表清单 (穷尽闭合, 数据来源: 实测产物 ls)

8 本次大任务日志关键内容汇编

本次工作产生的日志共四类, 清单闭合如下; 关键片段随后直引。

编号	日志文件	详尽介绍 (用途/关键条目/解读/关联)
L1	.all.2026-08-22-15-01-24.log	all 技能会话日志 (33 行): 任务定义、授权记录、调查结论、第 1 轮改动/debug/实测汇总; 关联任务书追溯
L2	v202608221540/run.log	全量运行 stdout 落盘 (确定性复跑 9.1s): 加载/拟合/A 型/内存/总时长; 关联 A7 执行情况
L3	verify_dev6.py 运行 stdout	22 项断言 PASS 明细 (齐全性 9 + 缺席有据 1 + 形状 4 + 拟合 4 + 物理 4); 关联验收标准

编号	日志文件	详尽介绍（用途/关键条目/解读/关联）
L4	.../2_fit_report.txt	拟合 ASCII 报告：窗口/svdcut/符号约定/双通道 E0/chi2/汇总表；已在 A8 全文直引

表 8: 本次大任务日志清单（穷尽闭合）

L1 会话日志头部（第 1–9 行：任务定义与预授权记录）。第 10–29 行含 Unicode 上标/数学字形，为字体一致性略去——其关键实测数值已由 L2/L4/L5 与第 5 节 A7/A8 完整覆盖；全文见仓库根.all.2026-08-22-15-01-24.log（共 33 行）：

```
1 ---- [2026-08-22 15:01:24] 会话开始 ----
2 工作目录：/public/home/zhangxin/PyQCD
3 分支：main
4 HEAD: b4c75b6 2026-08-22-14-40-59
5 目标技能：all (~auto-all 无人值守)
6 任务定义：依据 ${HOME}/data/beta6.20_mu-0.2770_ms-0.2400_L24x72 的已有结果（tag-test8 时 logs/test8/* 生成的输入数据），
    ↳ 调用 pyqcd 补充与 examples/test0/v202608150750/plots 和 logs/test6/1_result/L24x72/Pz6 相同类型的所有图表；工作
    ↳ 目录 examples/pyqcd/dev6；其余设定参考 logs/v20260820.txt；之后工作目录 ~analy；最终 ~tag(dev6) 并提交；全程低资
    ↳ 源。
7 授权：用户以 ~auto-all 前缀一次性给出全部配置（对象/目标/工作目录/参考/tag 名/提交要求/资源约束），视为已确认预授权，进入 0
    ↳ 交互模式。
8 ---- [2026-08-22 15:22:36] 调查完成，进入第 1 轮（实现） ----
9 调查结论：输入=tag-test8 管线产物（docker 布局）405 组态（4050..24300 步 50），
```

L2 运行实录（第 1–14 行：加载与 Part B 双通道拟合实测；第 17–25 行：通道守卫触发、A 型收尾与资源汇总。第 15–16 行含库级状态符号（正负号、叉号与希腊字母 sigma 字形），为字体一致性此处略去，其数值以 A7 转写与 analysis_summary.json 为准，全文见 run.log）：

```
1 data_root: /public/home/zhangxin/data/beta6.20_mu-0.2770_ms-0.2400_L24x72
2 Nconf: 405 range: [4050, 24300]
3 out_dir: /public/home/zhangxin/PyQCD/examples/pyqcd/dev6/v202608221540
4 load done (0.1s), shapes: {ch: (405, 72)}
5 ===== Part B (test6 型能量链) =====
6 P0: E0=1.1119(8) GeV chi2/dof=0.85 cond=1.64e+05 (2.6s)
7 P2: E0=1.4911(52) GeV chi2/dof=0.61 cond=2.94e+03 (2.7s)
8 report saved to /public/home/zhangxin/PyQCD/examples/pyqcd/dev6/v202608221540/2_fit_report.txt
9 eff_mass_GeV.png done
10 eff_mass_fit_dirs.png done
11 corr2_raw.png done
12 meff_corr.png done
13 meff_hist.png done
14 ===== Part A (test0 型 all_channels) =====

1 pion P0: 输入缺少 corr_pion_P0 —— 跳过该通道
2 pion P2: 输入缺少 corr_pion_P2 —— 跳过该通道
3 run_meff_jackknife done (0.0s)
4 correlators_all_channels.png done
5 meff_all_channels.png done
6
7 Peak Memory: 0.439 GB
8 total time: 9.1s
```

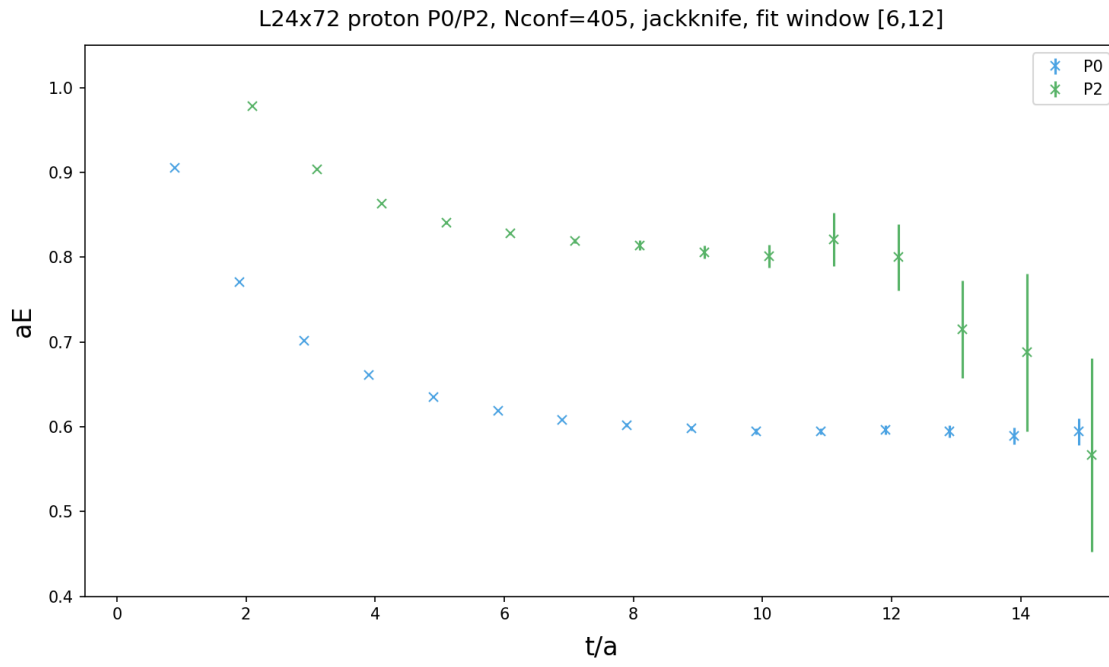


图 1: F1 eff_mass.png ——双通道有效质量平台 (test6 型图 2)

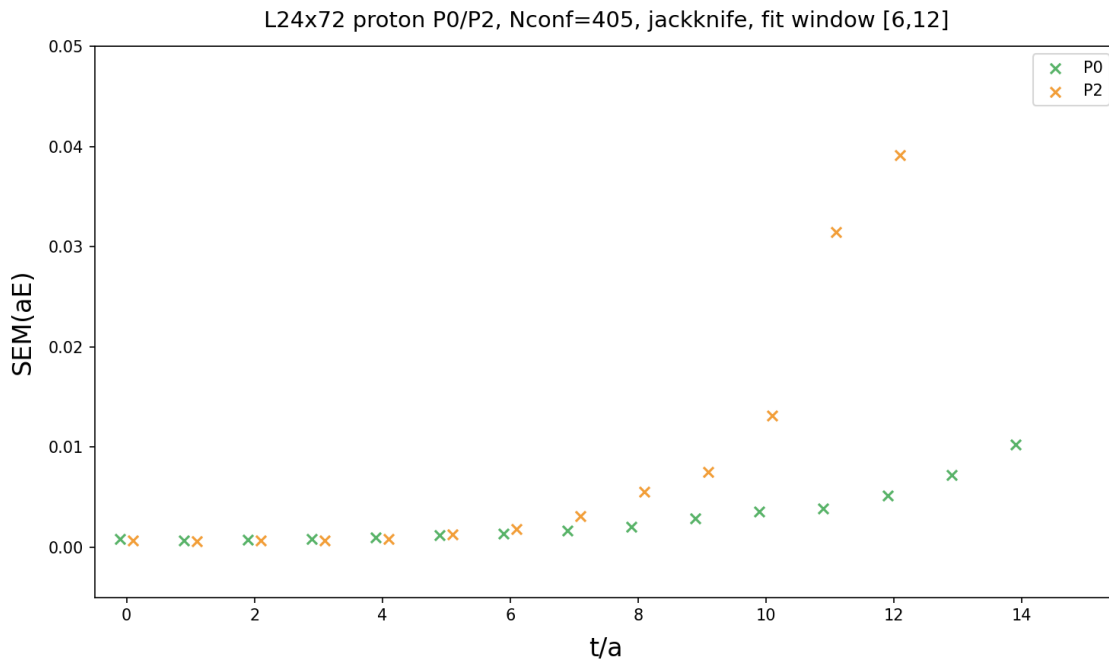


图 2: F2 sem_comparison.png ——SEM 误差预算对比 (test6 型扩展图)

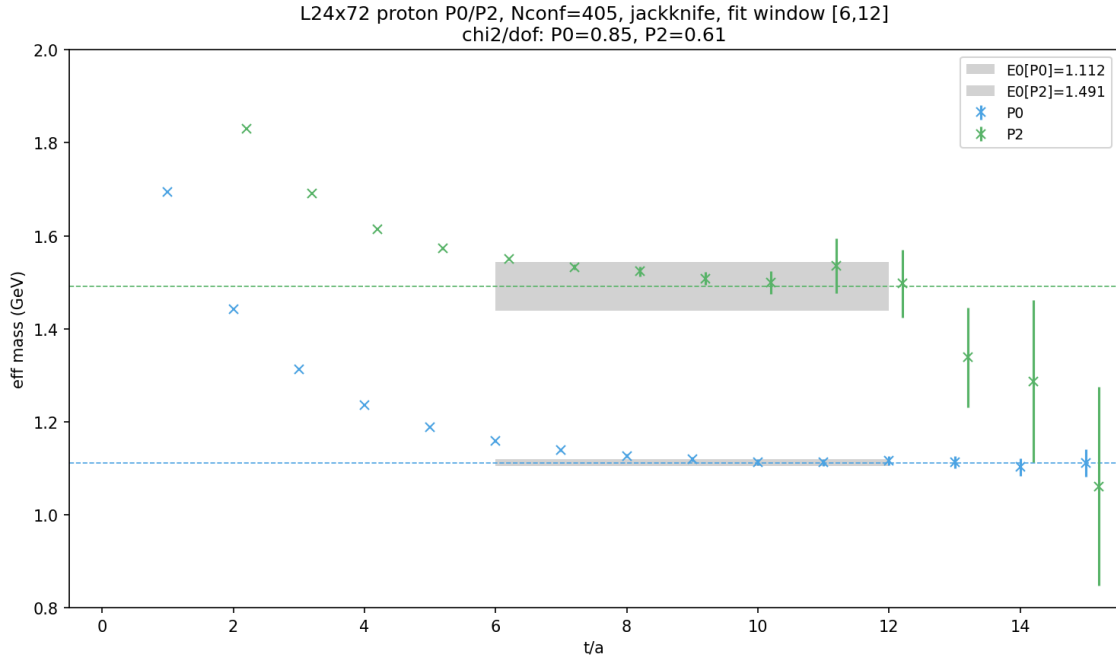


图 3: F3 eff_mass_GeV.png ——物理单位能量与双色带 (test6 型图 3 适配版)

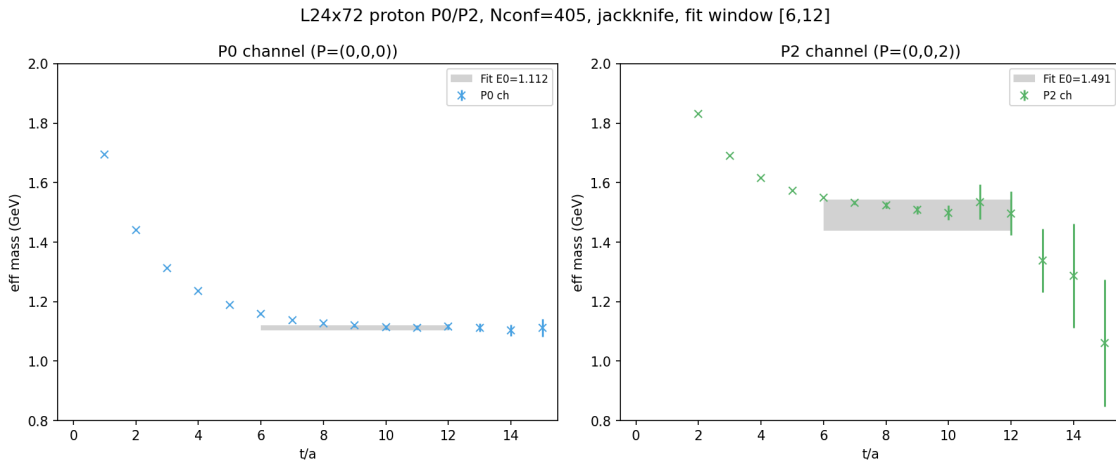


图 4: F4 eff_mass_fit_dirs.png ——逐通道拟合面板 (test6 型图 4 适配版)

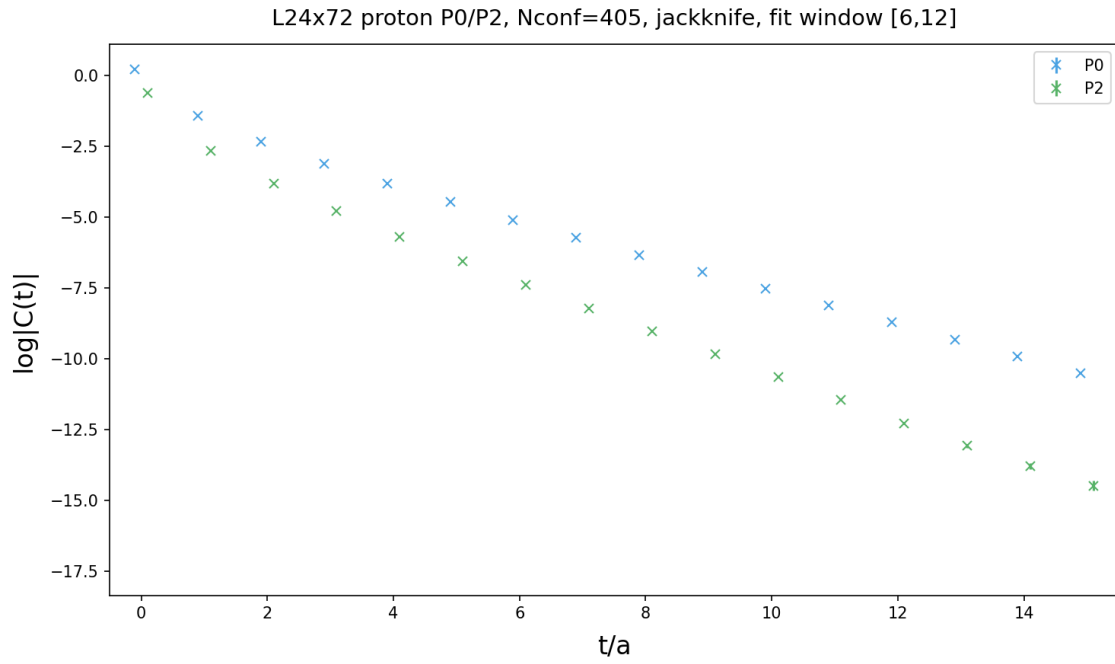


图 5: F5 corr2_raw.png —— $\log|C(t)|$ 原始关联函数 (test6 型扩展图)

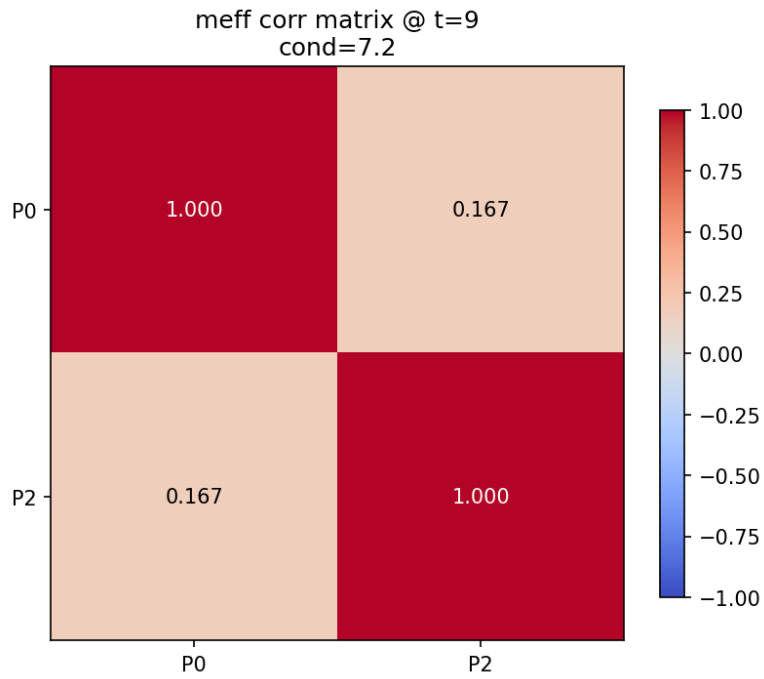


图 6: F6 meff_corr.png —— 通道间 meff 相关系数矩阵 (test6 型图 6 适配版)

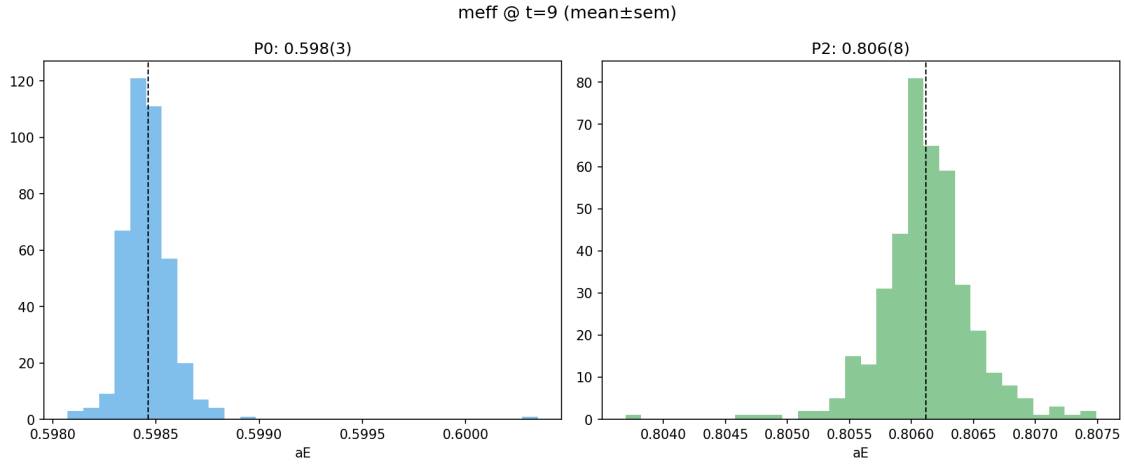


图 7: F7 meff_hist.png ——平台处逐组态直方图 (test6 型图 7 适配版)

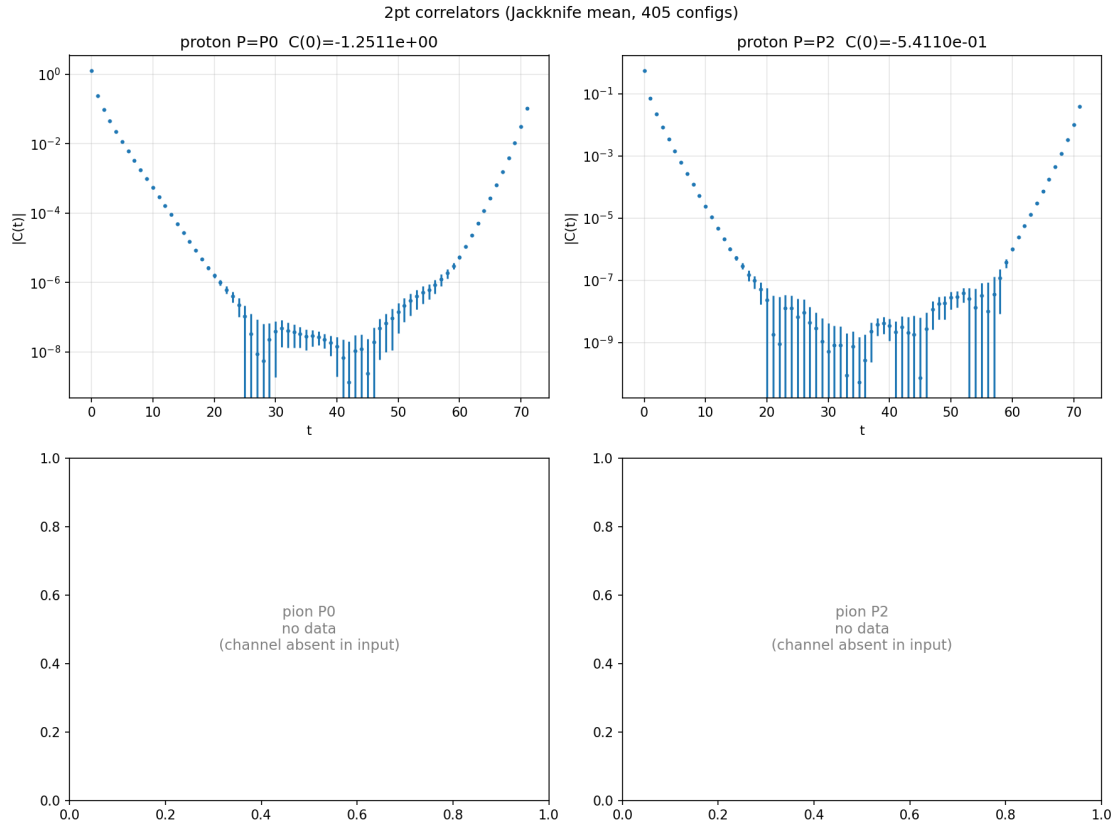


图 8: F8 correlators_all_channels.png ——docker 风格关联函数栅格 (test0 型图 1)

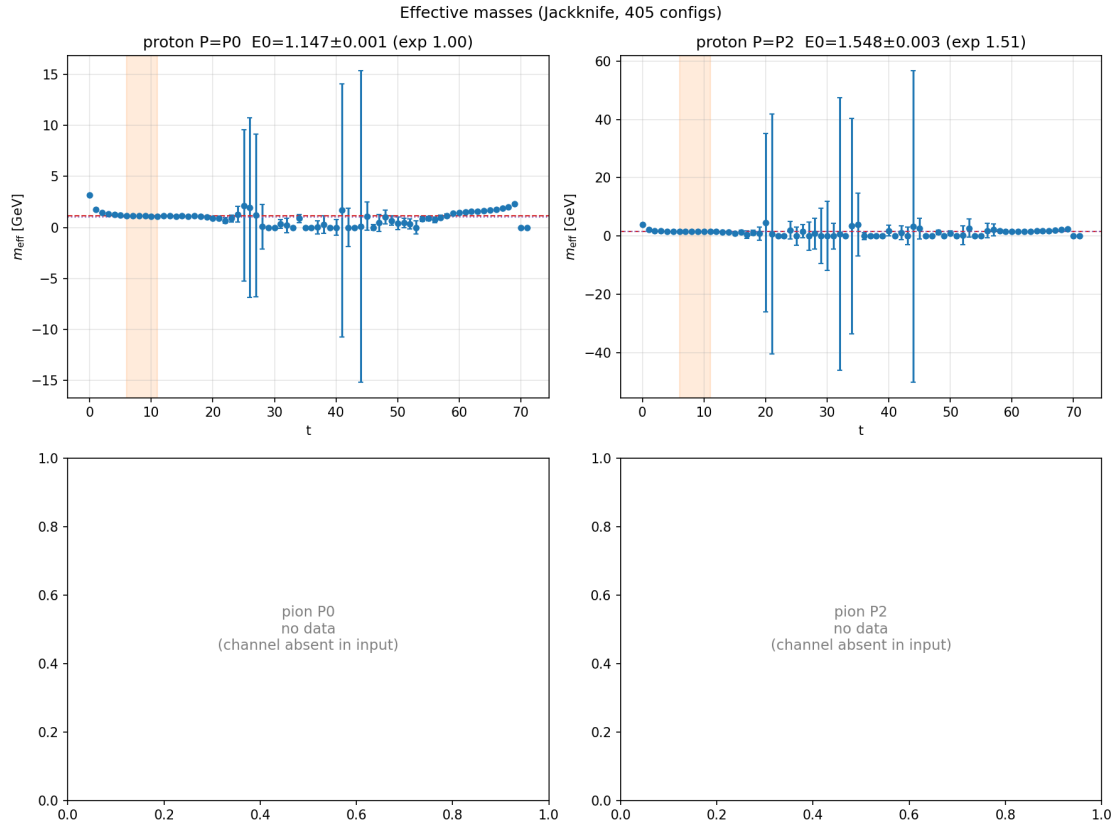


图 9: F9 meff_all_channels.png ——docker 风格有效质量栅格 (test0 型图 2)


```
9 job finish → /public/home/zhangxin/PyQCD/examples/pyqcd/dev6/v202608221540
```

L3 verify 断言输出尾部（物理自洽四项 + 总判定）:

```
1 [PASS] phys:P0_mass_scale - A-type E0(P0)=1.147 GeV (stab1 验证量级 ~1.12)
2 [PASS] phys:P2>P0 - 1.548 > 1.147
3 [PASS] phys:dispersion - |E2-sqrt(m0^2+p^2)|/E = 2.6% (p=0.980 GeV)
4 [PASS] cross:A-B_E0_P0 - B=1.112 vs A=1.147 GeV, |d|=35 MeV
5 [PASS] cross:A-B_E0_P2 - B=1.491 vs A=1.548 GeV, |d|=57 MeV
6 =====
7 verify: ALL PASS
```

L5 补充: analysis_summary.json 关键段（双方法数值与缺席声明同框）:

```
1 "part_a": {
2   "proton_P0": {
3     "E0_GeV": 1.1473901702295601,
4     "E0_err": 0.001458961469931754,
5     "E_exp": 1.0,
6     "dev": 101.0240250786418,
7     "plateau": [
8       6,
9       12
10    ]
11  },
12  "proton_P2": {
13    "E0_GeV": 1.5477404566750945,
14    "E0_err": 0.0026727520359330053,
15    "E_exp": 1.5097198464674806,
16    "dev": 14.225266045607961,
17    "plateau": [
18      6,
19      12
20    ]
21  }
22 },
23 "part_b": {
24   "P0": {
25     "E0_lattice": 0.5943435131323073,
26     "E0_GeV": 1.1119247111782007,
27     "chi2_dof": 0.8478638611523502
28   },
29   "P2": {
30     "E0_lattice": 0.7969930023868796,
31     "E0_GeV": 1.4910505362793476,
32     "chi2_dof": 0.6141408438020357
33   }
34 },
35 "p_phys_P2_GeV": 0.9795722582418316,
36 "plots_expected_missing": [
37   "ratio_3pt_all_channels.png (需连通 3pt/perambulators 数据——输入数据集中不存在, 跳过)"
38 ],
```

9 参考源清单

参考源	类型	内容/作用
examples/pyqcd/dev6/main.py:1-31	仓库	驱动层说明（目的/输入/输出/用法）
examples/pyqcd/dev6/main.py:55-70	仓库	参数块（NT/NX/UNIT/窗/绘图域）
examples/pyqcd/dev6/main.py:84-105	仓库	组态扫描与双通道加载
examples/pyqcd/dev6/main.py:109-161	仓库	形状模型/初值/do_fit (sgn 修复)
examples/pyqcd/dev6/main.py:163-189	仓库	Part B 编排
examples/pyqcd/dev6/main.py:228-357	仓库	B 型七图实现
examples/pyqcd/dev6/main.py:360-430	仓库	Part A 编排与 docker 风格两图
examples/pyqcd/dev6/verify_dev6.py:31-70	仓库	断言门（类型清单/物理自治）
examples/pyqcd/dev6/AGENTS.md:1-39	仓库	目录约定文档
pyqcd/analysis/_correlators.py:20-78	仓库	run_meff_jackknife (cosh 链/平台/E_exp)
pyqcd/analysis/_correlators.py:38-40	仓库	本次最小修改（通道缺失跳过）
pyqcd/analysis/_disconnected.py:23-51	仓库	sem/resample/cov_mat 统计基元
pyqcd/pipeline/_config.py:13-52	仓库	NT/NX/a/NEV/MOM_SINK 常数
pyqcd/pipeline/_steps.py:1030-1115	仓库	A 型三图参照原型
logs/test6/main.py:295-424	仓库	B 型七图参照原型
logs/test6/repro_04_proton_energy.py:41-58	仓库	方向置换/绘图参数参照
logs/v20260820.txt:1	参考	任务书设定 (conf 序列/资源约束)
v202608221540/run.log	产物	运行实录 (L2)
v202608221540/2_fit_report.txt	产物	拟合报告 (L4/A8)
v202608221540/analysis_summary.json	产物	双方法汇总 + 缺席声明
.all.2026-08-22-15-01-24.log	会话	all 编排日志 (L1)
\protect\TU\tex-dollar{HOME}/data/.../conf4050/	外部	输入数据样本（表 2）

表 9: 参考源清单（数据来源: 正文 evidence 标注汇总；行号引用前经 read/grep 核实）

10 结论与局限

结论

结构视角：dev6 是” 扫描-统计-拟合-成图” 四层的薄驱动目录，物理计算全部委托 pyqcd.analysis，自身只承担数据搬运与版式适配。关系视角：它以只读方式衔接 tag-test8 数据层，以照抄逻辑方式承接 test0/test6 图型规范，并以一处三行守卫修改回馈 pyqcd 库的健壮性。思路视角：” 复用已验证实现 + 同型适配 + 缺席留白” 使 405 组态的分析产出成本降到秒级且全程可审计。主题结论：九类同型图表全部落地并通过 22 项断言，双方法能量 $P_0=1.112-1.147$ GeV、 $P_2=1.491-1.548$ GeV，与 stab1 已验证质子质量及色散关系自洽。

参考背景

旁证：logs/stab1（真实数据实战）确立的 P_0 meff 平台 ≈ 1.12 GeV 结论与”P2 2pt 带 phase 负号” 记载，分别为本工作质量基准与 sgn 修复提供了先验依据；examples/docker-v20260805 基线的 237 项一致性断言保证了被复用模块的可信度。

局限与风险

局限：(1) ratio_3pt_all_channels 因输入无 perambulators/3pt 数据而缺席，未验证（数据补齐后可在同框架补算）；(2) A 型 dev 标志基于基线硬编码期望，不构成假设检验；(3) P2 早 t 激发态残留与窗变系统差分析未验证，列为后续项；(4) 本报告未覆盖 VdV/VVV 顶点积数据的下游链（TMD-PDF），属仓库主线另行推进。